

36118 Applied Natural Language Processing  
Master of Data Science and Innovation, UTS

## Assessment Task 2B

# End-to-End NLP Project — Technical Report

Sydney Liveability Explorer

**Subject** 36118 Applied Natural Language Processing  
**Submission** AT2 Part B — Group Technical Report  
**Due Date** Monday, 4 May 2026, 23:59  
**Suggested Length** 10–15 pages (excluding appendix)  
**Group** Group 3, Autumn 2026  
**GitHub Repository** <https://github.com/Nelkit/sydney-liveability-ai>

## Team Members

| Last Name     | First Name | Student ID | GitHub Username |
|---------------|------------|------------|-----------------|
| Chavez Calona | Nelkit     | 25609227   | @Nelkit         |

# Contents

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Executive Summary</b>                                       | <b>2</b>  |
| <b>2</b>  | <b>Project Objectives and Scope</b>                            | <b>2</b>  |
| <b>3</b>  | <b>Project Phases and Team Contributions</b>                   | <b>2</b>  |
|           | Phase Breakdown Table . . . . .                                | 2         |
| <b>4</b>  | <b>Data Sources</b>                                            | <b>2</b>  |
| 4.1       | Primary Data Sources . . . . .                                 | 2         |
| 4.2       | Data Limitations and Ethical Considerations . . . . .          | 3         |
| <b>5</b>  | <b>NLP Methods and Techniques</b>                              | <b>3</b>  |
| 5.1       | Text Preprocessing . . . . .                                   | 3         |
| 5.2       | Considered Approaches: Traditional NLP . . . . .               | 3         |
| 5.3       | Implemented Approaches: Modern NLP . . . . .                   | 3         |
| 5.3.1     | Reddit Data Collection via Arc Shift . . . . .                 | 3         |
| 5.3.2     | Aspect-Level Sentiment Analysis (DistilRoBERTa) . . . . .      | 3         |
| 5.3.3     | Sentence Embeddings (all-MiniLM-L6-v2) and ChromaDB . . . . .  | 5         |
| 5.3.4     | Multi-Agent Pipeline (CrewAI) . . . . .                        | 5         |
| 5.3.5     | Retrieval-Augmented Generation (RAG + LLM) . . . . .           | 6         |
| 5.4       | Justification: Why Modern Over Traditional . . . . .           | 7         |
| <b>6</b>  | <b>Findings and Evaluation</b>                                 | <b>7</b>  |
| 6.1       | NLP Pipeline Results . . . . .                                 | 7         |
| 6.2       | NLP Model Performance Metrics . . . . .                        | 7         |
| 6.3       | User-Facing Application Quality . . . . .                      | 8         |
| 6.4       | Temporal Trends . . . . .                                      | 8         |
| <b>7</b>  | <b>Outcomes and Value Added</b>                                | <b>8</b>  |
| <b>8</b>  | <b>Recommendations and Next Steps</b>                          | <b>8</b>  |
| <b>9</b>  | <b>Conclusion</b>                                              | <b>9</b>  |
| <b>10</b> | <b>References</b>                                              | <b>9</b>  |
| <b>A</b>  | <b>Appendix A: System Architecture Diagram</b>                 | <b>9</b>  |
| <b>B</b>  | <b>Appendix B: Dashboard Screenshots</b>                       | <b>9</b>  |
| <b>C</b>  | <b>Appendix C: Sample Code</b>                                 | <b>9</b>  |
| <b>D</b>  | <b>Appendix D: Evaluation Tables and Charts</b>                | <b>9</b>  |
| <b>E</b>  | <b>Appendix E: Traditional vs. Modern NLP Comparison Table</b> | <b>10</b> |

## 1 Executive Summary

Write this section last. Include one paragraph summarising each of the following: project objectives, data sources, NLP methods, findings, and outcomes.

## 2 Project Objectives and Scope

Describe the problem this project solves, the target user group, and what is in and out of scope for this submission.

## 3 Project Phases and Team Contributions

Describe project phases and map responsibilities to each team member. Each member must include an individual paragraph describing their specific contribution.

### Phase Breakdown Table

| Phase                          | Tasks                                                                                                                                                                | Team Member(s) |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| Phase 1: Data                  | Data acquisition and preprocessing from Reddit (Arc Shift), Community Insights PDF, BOCSAR CSV, TfNSW GTFS, OpenStreetMap (Overpass API), and City of Sydney ArcGIS. |                |
| Phase 2: NLP Pipeline          | Aspect-level sentiment analysis (DistilRoBERTa), sentence embeddings (MiniLM), ChromaDB ingestion, spaCy preprocessing.                                              |                |
| Phase 3: Multi-Agent Pipeline  | CrewAI pipeline crew (pre-computation) and query crew (real-time), FastAPI backend, RAG with Claude API.                                                             |                |
| Phase 4: Frontend              | Next.js dashboard, Leaflet map, aspect radar chart, emotion profile visualisation, conversational chat UI.                                                           |                |
| Phase 5: Evaluation and Report | NLP metrics, RAG quality, application evaluation, report, poster, peer review.                                                                                       |                |

## 4 Data Sources

### 4.1 Primary Data Sources

Document each source: name, access method, format, granularity, and contribution to the pipeline. Sources include: Community Insights PDF, Reddit via Arc Shift, BOCSAR CSV, TfNSW GTFS, OpenStreetMap via Overpass API, and City of Sydney ArcGIS.

## 4.2 Data Limitations and Ethical Considerations

Discuss Reddit as a public but self-selected source, privacy considerations, geographic bias toward inner Sydney suburbs, PDF granularity limitations, and the ethical use of community-generated content.

## 5 NLP Methods and Techniques

### 5.1 Text Preprocessing

Describe the preprocessing pipeline using spaCy: tokenisation, lemmatisation, and named entity recognition (NER) for suburb extraction. Also describe LangChain chunking strategy used for RAG ingestion.

### 5.2 Considered Approaches: Traditional NLP

Present a critical review of traditional NLP methods (VADER, LDA, TF-IDF) with reference to the literature. Explain why these approaches were considered but not adopted as the primary implementation for this project. Do not present them as implemented components.

### 5.3 Implemented Approaches: Modern NLP

#### 5.3.1 Reddit Data Collection via Arc Shift

Two complementary extraction paths feed a single per-suburb JSON shape under `data/processed/reddit/`. The first path uses `praw` against the live `r/sydney` API, issuing eight aspect-targeted keyword searches per suburb (safety, food and cafe, nightlife, affordability, transport, community, noise, green space). PRAW is rate limited and only reaches the most recent few hundred posts per query, so the second path uses the Arc Shift bulk dump, an NDJSON archive of historical `r/sydney` content. We partition Arc Shift records per suburb by string matching the suburb name and known variants against post and comment text. Both paths normalise to the same record shape (`text`, `suburb`, `score`, `created_utc`, `url`, `type`) so downstream code only handles one schema.

The pipeline currently covers 565 per-suburb extraction files in `data/processed/reddit/` and 563 matching analysis files in `data/processed/reddit_analyses/`. A floor of `score >= 2` removes the bulk of low-signal noise without dropping useful one-line residents' comments.

#### 5.3.2 Aspect-Level Sentiment Analysis (DistilRoBERTa)

The per-suburb analysis (`backend/core/nlp/pipeline.py`) runs four transformer-backed stages followed by a deterministic cross-modal fallback. Although the section heading retains the DistilRoBERTa label used in early planning, the implemented sentiment model is *yangheng/deberta-v3-base-absa-v1.1*, chosen because it scores polarity jointly with the aspect span instead of treating the whole text as a single signal.

**Aspect classification.** `backend/core/nlp/aspects.py` runs *facebook/bart-large-mnli* zero-shot classification over each chunk against eight liveability dimensions: safety, food and cafe, nightlife, affordability, transport, community, noise, and green space. A confidence threshold of 0.30 keeps

the top label; below that the chunk is tagged "general" and excluded from per-aspect aggregation. The same aspect catalogue drives ingestion tagging, search-time filters, and the chat agent's dimension routing, so changing it propagates consistently.

**Aspect-Based Sentiment Analysis.** `backend/core/nlp/sentiment.py` replaces the previously planned whole-text VADER scoring with DeBERTa-v3 ABSA. ABSA scores each (text, aspect) pair separately, so a single Reddit post saying *"the food is great but rent is brutal"* registers positive food and cafe sentiment and negative affordability sentiment instead of a neutral average. Polarity maps to a 0–1 scale where 0 is negative, 0.5 is neutral, and 1 is positive. Two routing rules guard against ABSA's known weaknesses on short, noisy text:

- A confidence floor of 0.55 on the ABSA softmax. The model is overconfident on terse Reddit one-liners, and below this floor the (text, aspect) pair is routed to a BART-MNLI fallback path.
- A word-count floor of 10. Texts shorter than this skip ABSA entirely and use the fallback path. The fallback contribution is also dampened in the per-suburb aggregate so a corpus of short posts cannot drown out the longer ABSA-confident signal.

**Coverage detection.** `backend/core/nlp/coverage.py` answers a different question from classification: not *how is the topic discussed?* but *is the topic discussed at all?* Each dimension has a hand-written prototype sentence (for example, *"public transport, trains, buses, and commuting around the area"*). We embed every chunk and the prototypes with the shared MiniLM encoder, take the top-10 cosine similarities per dimension, and tier the average per Table 2. Tier thresholds were tuned against a 30-suburb hand-labelled sample. This separation matters because zero ABSA pairs can mean two things: nobody talks about the topic, or the BART-MNLI top label landed on a different aspect. Coverage distinguishes the two cases honestly.

Table 2: Coverage tier thresholds (mean of top-10 cosine similarities to the dimension prototype).

| Tier   | Threshold                      | Source field       | Meaning                                                      |
|--------|--------------------------------|--------------------|--------------------------------------------------------------|
| strong | mean $\geq 0.40$               | reddit             | enough Reddit signal to score the dimension directly         |
| weak   | $0.25 \leq \text{mean} < 0.40$ | reddit or fallback | some chatter; score with reduced confidence                  |
| none   | mean $< 0.25$                  | none or fallback   | no Reddit signal; route through cross-modal fallback or null |

**Emotion profile.** `backend/core/nlp/emotions.py` runs DistilBERT fine-tuned on GoEmotions to attach a 27-label emotion distribution per chunk (anger, joy, sadness, fear, gratitude, and so on). The chunk-level distributions are aggregated to a per-suburb softmax that the chat agent uses for narrative hooks. GoEmotions is lightweight relative to ABSA, so it runs unconditionally on every chunk.

**Cross-modal fallback.** `backend/core/nlp/fallback.py` is the contract for what to do when Reddit is silent on a dimension. `FALLBACK_POLICY` is a static per-dimension dispatch table rather than a learned policy, which lets the report show the routing verbatim. Safety routes to BOCSAR crime-statistic percentile anchors; transport and walkability route to OSM scores from `data/processed/osm_scores`

demographics-leaning dimensions route to City of Sydney ArcGIS suburb rows. When neither Reddit nor a fallback produces a signal, the dimension's score is null and source is "none", and the chat agent treats the pair as missing data rather than fabricating a value.

**Output schema.** The pipeline writes one JSON file per suburb under `data/processed/reddit_analyses/`. Each `SuburbAnalysis` carries the suburb name, post count, fetch timestamp, an `aspects` dict keyed by dimension (with score, mentions, confidence, coverage, and source per entry), an emotions softmax over 27 GoEmotions labels, an LLM-generated narrative summary, and a list of curated quote sources. The explicit null score plus the coverage, source, and confidence trio are what let the chat agent verbalise an honest *"I don't have transport data on Abbotsbury"* response instead of inventing a number.

### 5.3.3 Sentence Embeddings (all-MiniLM-L6-v2) and ChromaDB

`backend/db/chromadb.py` manages the `sydney_liveability` collection, currently around 72,000 chunks built by two ingestion scripts. Embeddings come from `sentence-transformers/all-MiniLM-L6-v2`, accessed via a process-wide singleton in `backend/core/embeddings.py` so the coverage detector and the ChromaDB writer load the model exactly once (saving roughly 500 MB of repeated initialisation). The MiniLM choice trades a small amount of retrieval quality for a 5x smaller index and faster ingestion than larger encoders, which matters when the index doubles every time a new suburb's analyses are committed.

**Reddit ingestion** (`backend/scripts/ingest_reddit.py`) reads the per-suburb JSON arrays, splits each post with a 200-token chunker (20-token overlap) implemented in `chunk_reddit_text`, classifies each chunk's top liveability dimension via BART-MNLI (threshold 0.30, else "general"), and upserts into ChromaDB with `source="reddit"`. Chunk IDs are deterministic on (`source`, `post_id`, `chunk_index`) so re-running the script replaces rows in place rather than duplicating them.

**Sentiment ingestion** (`backend/scripts/ingest_sentiment.py`) ingests the per-suburb narrative summaries plus the curated quote chunks from `SuburbAnalysis.sources`, tagged `source="sentiment"`. The chat agent prefers these when looking for clean, attribution-friendly citations, falling back to the bulk Reddit chunks when none match the question.

Each chunk carries metadata for suburb, dimension, source, url, and score, so all retrieval filters run server-side in ChromaDB rather than as a Python post-filter on a wide query result.

### 5.3.4 Multi-Agent Pipeline (CrewAI)

The system uses two CrewAI crews with distinct responsibilities. The *pipeline crew* runs the per-suburb NLP analysis described above and writes `SuburbAnalysis` files; this is offline and runs whenever a suburb's Reddit corpus is refreshed. The *query crew* (`backend/crews/query_crew.py`) responds to chat turns in real time and is the focus here.

The query crew is sequential: a router, three specialists, an optional comparator, and a synthesiser. The router classifies the question into one or more of `crime`, `sentiment`, `gis`, `comparator`, or `out_of_scope`, and extracts the suburbs mentioned. For each non-out-of-scope route the orchestrator runs the matching specialists per suburb. The sentiment specialist (`backend/agents/query/sentiment.py`) is the agentic retrieval entry point and is detailed in the next subsection. When two or more suburbs are mentioned and `comparator` is in the categories, the comparator runs to produce a side-by-side ranking. The synthesiser composes the final answer from all specialist outputs.

Three retrieval tools are exposed to the sentiment specialist via `backend/core/agent/tools.py`. `search_posts(suburb, dimension, query, k)` performs dense semantic search over the `sydney_liveability` collection filtered by suburb (required) and dimension (optional). `get_suburb_aspect(suburb, dimension)` is a structured lookup against the cached `SuburbAnalysis`. `compare_suburbs(suburbs, dimension)` sorts an input list by `aspects[dimension].score` descending and drops null-scored entries, refusing inputs longer than ten suburbs to prevent the agent from re-ranking the global corpus through this entry point. All three tools refuse to fabricate when the cached aspect is null-scored: they return `{status: "no_data", reason: ...}` rather than empty results, which the agent can verbalise honestly.

The sentiment specialist runs a deterministic mini-pipeline rather than an open ReAct loop, because CrewAI does not expose a graph-node abstraction equivalent to the LangGraph design that originally specified ReAct. For each suburb the specialist (a) calls `get_suburb_aspect` once per dimension to build the structured aspect payload, then (b) issues at most one question-driven `search_posts` call routed to the dimension whose keywords best match the question. The hard ceiling is five question-driven calls per turn; the structured fan-out is unbounded because it is a deterministic tour of the eight dimensions, not an agent decision.

### 5.3.5 Retrieval-Augmented Generation (RAG + LLM)

Every tool call the sentiment specialist makes is recorded as a `TraceEntry` in an `evidence_trace` list, with the shape `{step, tool, arguments, reasoning, result_count, result_preview, elapsed_ms}`. The trace is the audit surface for the whole retrieval-augmented turn. The synthesiser receives the flattened trace alongside the structured specialist outputs, and is instructed to cite at least one trace entry per named-suburb claim about resident sentiment. When the synthesiser falls back to database-only context (no Reddit signal for the matched suburbs), it omits the Evidence trace: block from its prompt rather than emit an empty heading, so the LLM does not learn to hallucinate citations to nothing.

A spatial-intent gate keeps the trace block out of out-of-scope turns. The synthesiser checks `agent_outputs["router"]["categories"]`; when the categories equal `["out_of_scope"]` the prompt drops the trace block entirely and the answer reverts to a polite refusal. The check is keyed off the canonical router output rather than the presence of a sentiment specialist, so future routing changes do not silently re-introduce stale citations.

The `POST /api/chat` response also gains an optional quality field carrying `evidence_trace_summary`: a deterministic aggregate over the per-suburb traces with `length`, `by_tool` counts, `last_action`, and `no_data_count`. The aggregate is pure Python (no extra LLM call, no network round-trip), so it adds no per-turn latency. It is optional and additive: existing frontend clients that read only `answer`, `sources`, `suburb_scores`, and `map_state` remain compatible. A future debug panel can render the agent's actual decision path from this field without backend changes.

The chat LLM itself is configurable per agent via `LLM_AGENT_MODELS_JSON`. The default routes through OpenRouter for cost during development, with overrides to *anthropic/claude-3-5-sonnet* for production-quality synthesis. The synthesiser's prompt is constructed in `backend/agents/query/synthesiser.py` and includes the question, a database-derived context block (facilities, transport, OSM amenity counts), the structured specialist outputs, and the evidence trace.

## 5.4 Justification: Why Modern Over Traditional

Present an academic argument for each design choice: DistilRoBERTa over VADER, sentence embeddings over TF-IDF, RAG over keyword search, and CrewAI for modular orchestration. Cite Lewis et al. (2020), Reimers & Gurevych (2019), and Hutto & Gilbert (2014).

## 6 Findings and Evaluation

### 6.1 NLP Pipeline Results

Summarise aspect scores per suburb, emotion profiles, transport scores, POI density, and crime trends. Reference Appendix D for full tables and charts.

### 6.2 NLP Model Performance Metrics

The chat hot-path emits a deterministic `evidence_trace` on every turn and a small `quality.evidence_trace_summary` aggregate on the response payload, which is enough information for a future debug panel without an inline LLM judge slowing the user down. Grading retrieval quality happens out of band in `backend/scripts/run_agent_eval.py`, an offline LLM-judge evaluator that produces the per-prompt verdicts this section quotes.

The script runs the following loop:

1. Load a curated 30-prompt set from `data/eval/prompts.yaml`, composed of 10 spatial-grounded prompts, 10 no-data prompts (null-scored aspects or unknown suburbs), 5 multi-suburb comparison prompts, and 5 out-of-scope prompts.
2. Run each prompt through `crews.query_crew.run_query`, capturing the answer, the per-suburb evidence trace, and the wall-clock latency.
3. Ask one LLM judge per prompt, using a single uniform template across all categories: *“does every named-suburb claim in the answer have a supporting trace entry? Reply JSON: {retrieval\_grounded, reason}”*.
4. Write `data/eval/results.csv` with one row per prompt, replacing rather than appending so re-runs produce a deterministic snapshot.
5. Print a final summary: `N prompts: K grounded, M ungrounded, P agreement-with-prior`.

The CSV schema is reproduced in Table 3. The judge model is configurable via the `-judge-model` CLI flag, falling through `LLM_AGENT_MODELS_JSON["evaluator"]` and the global `LLM_MODEL`; the chosen model name is recorded per row so the report can compare verdicts across, for example, a cheap iteration model and a stricter Opus-tier judge.

Table 3: Schema of data/eval/results.csv, one row per prompt.

| Column             | Type  | Source       | Description                                            |
|--------------------|-------|--------------|--------------------------------------------------------|
| id                 | str   | prompts.yaml | kebab-case prompt identifier                           |
| prompt             | str   | prompts.yaml | the user question                                      |
| answer             | str   | agent        | the synthesiser's response text                        |
| retrieval_grounded | bool  | judge        | every named-suburb claim is supported by a trace entry |
| expect_grounded    | bool  | prompts.yaml | the labeller's prior verdict for agreement scoring     |
| reason             | str   | judge        | the judge's free-text rationale                        |
| trace_length       | int   | agent        | total evidence_trace entries this turn                 |
| n_no_data          | int   | agent        | trace entries reporting no_data                        |
| judge_model        | str   | CLI          | model name used by the judge                           |
| agent_latency_ms   | float | agent        | wall-clock time of run_query() in milliseconds         |

The judge prompt is uniform across categories by design. A category-specific rubric (for example, “for no\_data prompts, grounded means the answer says I don’t have that data”) would create per-category bias, whereas one uniform criterion (“every named-suburb factual claim has a trace entry”) generalises cleanly: an out-of-scope refusal that names no suburbs is trivially grounded, an honest no-data answer with no fabricated claims is grounded, and a hallucinated suburb claim with no supporting trace is ungrounded. The judge’s full reasoning is logged per row, so the report can audit noisy verdicts manually instead of building a hand-labelled gold set.

**Known limitations.** LLM-judge verdicts on paraphrased citations are noisy; the curated 30-prompt set bounds generalisation claims to “directional retrieval-quality evidence on a curated set,” not a production benchmark; and the Arc Shift dump is a snapshot, so discourse drift after the dump date is not reflected in retrieval until the next bulk re-ingest.

### 6.3 User-Facing Application Quality

Describe the dashboard, aspect radar visualisation, emotion profile chart, chat interface, and overall user experience quality based on walkthroughs or testing.

### 6.4 Temporal Trends

Summarise time-based patterns in Reddit sentiment per suburb where timestamps are available.

## 7 Outcomes and Value Added

Describe what the project achieved and the practical value delivered to the target user group.

## 8 Recommendations and Next Steps

List the highest-impact extensions: coverage of more suburbs, additional data sources, model fine-tuning, and integration with rental price APIs.

## 9 Conclusion

Summarise the project contributions, lessons learned, and future potential.

## 10 References

Use APA 7th edition for all citations. Key references include: DistilRoBERTa (Sanh et al., 2019), all-MiniLM-L6-v2 (Reimers & Gurevych, 2019), RAG (Lewis et al., 2020), CrewAI, and Arc Shift.

### A Appendix A: System Architecture Diagram

Insert the full pipeline diagram: Arc Shift, GTFS, OSM, BOCSAR, and PDF ingestion feeding into DistilRoBERTa and ChromaDB, then CrewAI, RAG, and the frontend.

### B Appendix B: Dashboard Screenshots

Insert annotated screenshots of the aspect radar, emotion profile, Leaflet map, and chat interface.

### C Appendix C: Sample Code

Include representative code for `ingest_sentiment.py`, CrewAI agents, the RAG chain, and FastAPI endpoints.

### D Appendix D: Evaluation Tables and Charts

Include suburb-level scores per sentiment dimension, emotion distributions, transport scores, and POI density charts.

## E Appendix E: Traditional vs. Modern NLP Comparison Table

Table 4: Traditional vs. Modern NLP Approach Comparison

| Approach                              | Type        | Used | Rationale                                     |
|---------------------------------------|-------------|------|-----------------------------------------------|
| <b>VADER sentiment</b>                | Traditional | No   | Lexicon-based, no aspect granularity          |
| <b>LDA topic modelling</b>            | Traditional | No   | Unsupervised, low coherence on Reddit text    |
| <b>TF-IDF keyword extraction</b>      | Traditional | No   | Frequency-based, no semantic understanding    |
| <b>DistilRoBERTa aspect sentiment</b> | Modern      | Yes  | 8-dimension aspect scoring, contextual        |
| <b>all-MiniLM-L6-v2 embeddings</b>    | Modern      | Yes  | Semantic similarity, compact and fast         |
| <b>RAG with Claude Sonnet</b>         | Modern      | Yes  | Grounded, cited answers from retrieved chunks |
| <b>CrewAI multi-agent pipeline</b>    | Modern      | Yes  | Modular orchestration, scalable per suburb    |